Jordan University of Science and Technology

Faculty of Computer and Information Technology

Department of Computer Engineering

CPE 591: Graduation Project I Report

Grab&Go

Ahmad Edais (ID: 152716)

Malak Ajlouni (ID: 147258)

Mohannad Rababah (ID: 160393)

Laith Allabadi (ID: 163130)

Supervised By Dr. Moath Jarrah

Contents

1. Abstract 3
2. Background 3
2.1 The Retail Checkout Bottleneck 3
2.2 Research Motivation: Bridging the Accessibility Gap 3
3. Proposed Work 4
3.1 System Architecture 4
3.2 Person Tracking: Multi-View 3D Tracking with NVIDIA DeepStream and MV3DT 4
3.3 Item Detection: Proximity-Driven ROI Logic 5
3.4 Simulation & Synthetic Data Generation 6
3.5 Hardware & Spatial Configuration 8
3.6 Mobile Application & Backend System 8
3.7 Use Cases & User Interaction Flows 12
4. Professional Practice Constraints 13
4.1 Manufacturability & Deployability 13
4.2 Economic Constraints 13
4.3 Sustainability & Environmental Responsibility 14
4.4 Ethical Standards & Privacy-by-Design 14
4.5 Data Security & API Integrity 14
5. Deliverables 15
5.1 Simulation Environment (Blender Digital Twin) 15
5.2 Computer Vision & Tracking Module 15
5.3 Backend API & Database 16
5.4 Mobile Application (Flutter) 16
6. Team Roles & Responsibilities 17
7. Project Schedule 18
8. Bibliography 19

1. Abstract

Checkout procedures remain a critical bottleneck in traditional retail, with over 70% of shoppers identifying waiting at the register as their most frustrating in-store experience. While autonomous "just-walk-out" systems have emerged to address this inefficiency, commercial implementations (Amazon Go, Cloudpick) rely on expensive proprietary sensor fusion infrastructure—often exceeding $50,000 per installation—making them inaccessible for most markets.

Grab&Go proposes a vision-centric alternative using off-the-shelf hardware and open-source tools. The system has three main parts: (1) multi-camera person tracking using NVIDIA DeepStream SDK with MV3DT (Multi-View 3D Tracking) to keep track of who is who across camera views and deal with situations where people are blocking each other; (2) proximity-driven item detection using hand Region-of-Interest (ROI) logic and CV-based product classification; and (3) synthetic data generation using Blender to speed up model training without having to collect real-world data, which costs a lot of money. A mobile app built with Flutter takes care of the user's entire life cycle, from QR code entry to real-time cart visualization to automatic billing when they leave.

This report describes the technical architecture, the reasons for the design of each subsystem, and the simulated validation environment that shows it can be done before it used. The prototype works on an edge-cloud hybrid model, processing high-frequency vision tasks locally and sending authentication, billing, and analytics to a backend API. This keeps cart updates to about 500 milliseconds.

---

2. Background

2.1 The Retail Checkout Bottleneck

The traditional point-of-sale process is a major source of friction in modern retail. According to industry data, the amount of time people spend waiting at the checkout is directly related to how many people leave their shopping carts: 70.22% of shopping carts are abandoned worldwide [1], which costs businesses $18 billion in lost sales each year [2]. For a convenience store operating 16 hours daily with average transaction time of 3 minutes per customer, this bottleneck alone represents a 15–20% reduction in daily throughput.

There is a lot of economic pressure to deal with this right away. The global smart retail technology market is expected to grow from $52.1 billion in 2025 to $139.6 billion by 2030, with a compound annual growth rate (CAGR) of 21.8% [3]. This shows that industry is investing a lot in autonomous solutions.

2.2 Research Motivation: Bridging the Accessibility Gap

Recent CV research demonstrates scalable vision-only solutions for retail automation, outperforming RFID in cost and accuracy [4]. However, there is still a big problem: collecting labeled training data on a large scale is too expensive to do in real stores, which makes it hard to build strong models for tracking people and finding items.

Grab&Go fills this gap by:

- Getting rid of proprietary dependencies: Using NVIDIA DeepStream (an open-access SDK) instead of closed-source tracking engines.
- Making thousands of synthetic data that are very close to the real-world and do the job for training.

So this demonstrates our vision in making this service for everyone.

---

3. Proposed Work

3.1 System Architecture

Grab&Go operates on an edge-cloud hybrid model optimized for latency and cost:

Edge Side:

- Nvidia Deepstream SDK on Jetson AGX Orin or similar model.
- Real-time video processing: person tracking, item detection, spatial reasoning
- Motion-triggered inference.

Cloud/Backend Side:

- ASP.NET Core Web API
- SQL Server relational database
- JWT-based authentication

User Interface:

- Flutter cross-platform mobile application
- Real-time cart visualization via SignalR with fallback to RESTful polling
- QR-based entry and digital invoicing

3.2 Person Tracking: Multi-View 3D Tracking with NVIDIA DeepStream and MV3DT

3.2.1 Problem Definition: Why 3D Tracking Matters

In retail settings, standard 2D bounding box tracking doesn't work when one customer stands behind another. This makes the backend lose track of who they are for a short time, which could lead to Customer A's items being incorrectly assigned to Customer B. For autonomous checkout accuracy, this is not acceptable.

The NVIDIA DeepStream SDK with MV3DT (Multi-View 3D Tracking) combines data from several cameras to find the 3D global coordinates (X, Y, Z) for each customer being tracked.

3.2.2 The MV3DT Pipeline

Step 1: Multi-Camera Calibration

- Each camera's intrinsic parameters (focal length, principal point, distortion coefficients) and extrinsic parameters (rotation matrix R, translation vector T relative to a global origin) are pre-calibrated.
- In the physical prototype, this will be done via checkerboard calibration. In our simulation (Blender), we explicitly define these parameters.

Step 2: Person Detection & Feature Extraction

- Each camera stream has a lightweight person detector (like PeopleNet) that makes 2D bounding boxes for each frame.
- Re-Identification (Re-ID) takes features from people so that in case the person doesn't appear in any of the cameras it can re-identifies him using these features.

Step 3: Linking Cameras

- The system compares the Re-ID feature vector of each person found in Camera A to the Re-ID feature vector of each person found in Camera B.
- If cosine similarity is greater than the threshold (usually 0.6), they are the same person.
- The calibration matrices convert from a 2D image to the 3D world space.

Step 4: Assigning a global ID

- Each person gets his ID the moment he scans the QR, as each person in the database will have a unique ID.
- Using triangulation across multiple views, their 3D position is maintained+.

3.3 Item Detection: Proximity-Driven ROI Logic

3.3.1 The Problem with Full-Frame Detection

Running object detection on every frame for every customer is computationally expensive: for example, a single 1920×1080 frame processed by YOLO26m takes 4.5 ms, infeasible when tracking 30+ simultaneous customers across 10 camera views.

3.3.2 Proximity-ROI Logic

We implement a **conditional detection framework**:

IF (Customer_3D_Position intersects Shelf_B_Volume) THEN
Trigger item-detection inference on Shelf_B region only

ELSE
Skip for Shelf_B

This dramatically reduces compute load. For a 4-shelf store with 3 simultaneous customers, we process only 3 ROIs per frame instead of 12.

3.3.3 Item Interaction Recognition

Once the model detects products in the active ROI, we track item state transitions:

- **Pick Event:**
    - Frame T-1f: Hand was empty + hand velocity indicates motion into shelf_A.
    - Frame T: Customer's hand enters shelf_A + hand velocity indicates motion away from shelf_A + Hand appears holding something.
    - Event logged: {Customer_ID: "C_001", Item_ID: "Tuna_Sayyad", Action: "Pick", Timestamp: 1642345800}
- **Return Event:**
    - Frame T: Customer's hand enters Shelf_A, where the item they were holding comes back and then leaves empty.

Hand position is derived from the customer's 3D centroid (from MV3DT) and hand-centric pose estimation using OpenPose or MediaPipe. In the simulation, we annotate hand keypoints directly.

3.4 Simulation & Synthetic Data Generation

3.4.1 Motivation for Simulation

It's very difficult to collect data for people tracking and for every item in the store manually and then label them for training!

Solution: We use Blender to create a "Digital Twin" of the demo store, enabling:

1. Automatic generation of thousands of labeled frames with different conditions.
2. Validation of camera placement and spatial logic without drilling walls.

3.4.2 Blender Environment Design

The layout is a 2×3 meter rectangle (6 square meters).

- Four shelves on the sides, each 1.2m tall and 0.5m wide
- Entrance zone.
- Exit zone.

- Height of the ceiling: 2.8m (this lets cameras be mounted at 2.5m height for tracking without obstructions)

Virtual Products: 20 SKUs modeled at a 1:1 scale (like snack boxes and beverage bottles) and UV-mapped with realistic textures.

Setting up the camera:

- Camera 1 (wide view): covers a wide area from the market **(**2.5m), Captures full-body shapes to check that people are being tracked correctly. This type of cameras' views have to overlap.
- Camera 2 (on shelf): Same specs, but at a 30° angle to Shelf A. It helps people who are blocked by Camera 1 to be seen again.

We chose these settings to mimic real-world deployment limits: cameras mounted on the ceiling at standard CCTV heights.

3.4.3 Synthetic Data Generation Pipeline

- We need to randomize the environmental parameters to cover all cases in the real-world, like lightning, camera jittering.
- Then we will render a frame each like 500ms, and then send this frame to a function that draws bounding boxes around objects/people.
- Save the frame with COCO-format labels.
- Iterate over this process as much as we need to get sufficient labeled data.

3.4.4 Testing Possible Scenarios

Scenario A: Not being able to make a choice

- The customer picks up an item, holds it for three seconds, and then puts it back on the shelf.
- Test: Does the system correctly log "Pick" and "Return" events with the right timestamps?

Scenario B: A busy checkout line

- Two customers walk close to each other which makes the aisles look crowded.
- Test: Does Re-ID accurately identify them?

Scenario C: Changes in the lighting

- Blender makes the same scene look the same in 100 lux (dim), 500 lux (normal), and 800 lux (bright) light.

- Test: Does the object-detection model keep the same level of confidence in product detection no matter what the lighting is like?

The outcomes of these scenarios directly influence hyperparameter tuning (Re-ID similarity threshold, object-detection confidence cutoff) prior to physical testing.

3.4.5 Domain Randomization for Robustness

The synthetic dataset is purposefully varied:

- **Colors of Clothing:** 50 different colors (RGB values) randomly assigned to agents.
- **Different Poses:** Blender supports different poses so we will give each agent around 10 different poses.
- **Lighting:** different lighting setups so that we can handle cases like a lamp is damaged.

This variety makes sure that the object-detection model and Re-ID models trained on synthetic data can work with new real-world setups. This is called Domain Randomization [5].

3.5 Hardware & Spatial Configuration

3.5.1 Edge & Cloud Hardware Requirements

- **Testing Phase:** A mid-range laptop that has a Nvidia RTX GPU, enough RAM and Storage.
- **Production:** For the edge side we will use Nvidia Jetson boards, the exact model is not fixed as we-plan to make it dynamic based on the market size, load, number of SKUs etc…, same for the cloud's specs, as it varies a lot from a client to another.

3.5.2 Camera Hardware Requirements (Physical Deployment Phase)

- **Sensor:** 1/2.3" or larger.
- **Resolution:** ≥1080p (1920×1080).
- **Frame Rate:** 30 FPS.
- **Lens:** 35mm or 50mm fixed focal length (makes calibrating easier).
- **Networking:** Gigabit Ethernet.

3.6 Mobile Application and Backend System

3.6.1 User Login and Dynamic QR code Entry

The Flutter app uses **JWT-based stateless authentication**:

1. **User Registration:**
    - App sends POST request to /auth/register using an email and password.
    - The backend (ASP.NET Core Identity) hashes the password and saves it in SQL Server.
    - Returns 201 Created.
2. **Store Entry:**
    - User presses the "Enter Store" button.
    - App requests GET /qr/generate with a token from the user.
    - The backend generates time-sensitive QR code (valid for 30 seconds, embedded with user ID + timestamp + HMAC signature).
    - QR code is refreshed every 30 seconds, which stops replay attempts.
    - User scans QR code at the entrance; system logs the entry and starts the virtual cart.

3.6.2 Real-Time Cart Synchronization

Main Path (SignalR):

- Upon entry, the app connects to the SignalR Hub at /hubs/cart.
- Every item interaction detected by vision pipeline triggers a server-sent event.
- The app receives the event right away: {"action": "pick", "item": "Chips", "timestamp": 1642345800}.
- Cart UI updates with <100ms latency.

Fallback Path (RESTful Polling):

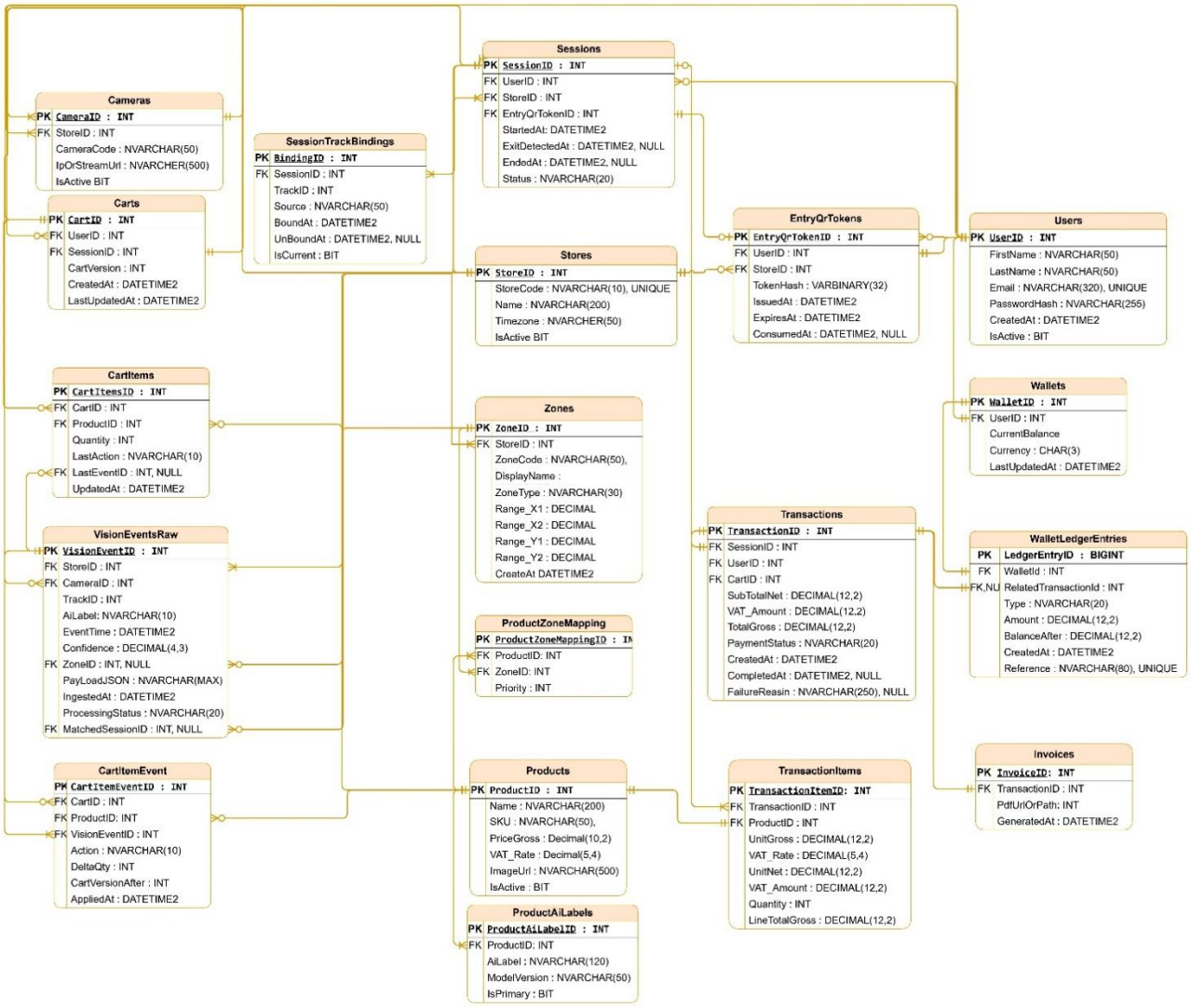
- If SignalR connection breaks, the app polls GET /cart/current every 2 seconds.
- Keeps data consistent even under poor network conditions.

Backend Cart Reconciliation:

- Cart state is versioned (which means every time you interact with an item, the cart version goes up).
- If client and server versions are different, the backend uses the vision pipeline's version as the truth (vision data is more reliable than potential app glitches).

3.6.3 Product Search & Data Management

Database Schema (3 NF Normalized):



Search Logic:

- User types "Chips" in Flutter app search bar.
- App sends GET /products/search?query=Chips.
- Backend executes parameterized SQL query:

  SELECT p.Name,p.PriceGross,p.ImageURL,z.ZoneCode,z.DisplayName

  FROM Products p

  INNER JOIN ProductsZoneMapping pzm

  ON p.ProductID = pzm.ProductID

```
INNER JOIN Zones z
ON pzm.ZoneID = z.ZoneID
WHERE
p.IsActive 1 AND z.StoreID = @StoreID AND p.Name LIKE @Query
```

- Returns product details + physical location (e.g., "Shelf 2, Position (2.1, 1.3)").

This approach leverages **Entity Framework Core** ORM to abstract SQL queries, reducing SQL injection risk.

3.6.4 Automated Billing & Invoice Generation

Exit Event Flow:

1. Vision system sees the customer leaving geofence (crossing Y > 5 meters toward the exit).
2. System logs the time of exit and sends POST request to /checkout/finalize with CartID.
3. Backend adds up cart items:
   Subtotal = SUM(product.price FOR each item in cart)
   Tax = Subtotal × 0.16 (Jordan VAT rate)
   Total = Subtotal + Tax
4. Transaction record created: Transaction record was created in SQL Server with the following information :TransactionID, UserID, CartID, Total and Status: "Pending".
5. Payment Processing: Payment is processed via an internal Digital Wallet system. The backend validates the user's wallet balance and then makes an atomic transaction to deduct the Total amount.
6. Completion: After the ledger update succeeds, TransactionStatus is updated to "Completed".
7. A backend service (QuestPDF Library) makes the digital invoice (PDF) and sends it to the app.
8. User can view the invoice on the Transaction History tab in the app.

Key Feature: No physical receipt is printed, aligning with sustainability goals.

3.6.5 Flutter App Architecture

State Management: Provider pattern with ChangeNotifier classes:

- UserProvider: Manages authentication state (logged-in user, JWT token).
- CartProvider: Listens to SignalR Hub events and updates local cart state.
- ProductProvider: Stores product catalogs and search results.

UI Screens: (Appendix B)

- **Login/Register:** Email + password authentication.
- **Home:** Shows all available stores on display.
- **Store page:** Store details and QR code reader button for entry.
- **Live Cart:** Real-time cart display (synced via SignalR).
- **Transaction History:** Past invoices and receipts.
- **Settings:** Theme toggle (light/dark), language selection, budget limits.

Network Layer:

- Dio package for REST API calls, which come with a JWT token for each request.
- **signalr_netcore** package for **SignalR Hub** connections.
- Error handling: Automatic retry logic (exponential backoff) for temporary failures.

3.7 Use Cases And User Interaction Flows

The Use Case diagram (Appendix A) show what the system needs to be able to do:

Primary Actors: Customer (Shopper), System (Backend + Vision Pipeline).

Main Use Cases:

1. **Register Account:** Customer provides email, password; system stores credentials securely.
2. **Generate Dynamic QR:** Customer asks for an entry token, and the backend generates time-sensitive QR code.
3. **Login:** Customer authenticates via email + password; system grants JWT token.
4. **Change Theme/Language:** The customer changes the app's preferences (light/dark mode, English/Arabic).
5. **Set Budget Limit:** Customer sets a spending limit, and the system alerts them if cart exceeds that limit.
6. **View Purchase History:** Customer reviews past transactions and invoices.
7. **Search Products:** Customer searches for product name, the system returns available items.
8. **View Item Details:** When a customer taps product card, the system shows price, description, stock status.
9. **View Real-Time Cart:** Customer sees live cart updated as items are picked/returned.
10. **View Total Bill:** Customer sees subtotal, tax, and total before leaving.
11. **Receive Invoice:** When the customer leaves the store, he receives a digital invoice via app.

12. **Scan QR Code (System):** Vision system detects customer entry via QR scan, initializes session.
13. **Track Customer:** Vision pipeline keeps track of a customer's identity via MV3DT.
14. **Detect Item Interaction:** Vision system recognizes pick/return events via hand-ROI logic.
15. **Process Wallet Transaction:** System completes the transaction and charges the customer when they leave the store.

---

4. Professional Practice Constraints

4.1 Manufacturability & Deployability

Hardware Standardization:

- All of the cameras are of the-shelf commercial ones.
- No need for custom hardware.

Software Portability:

- Backend and vision modules are containerized via Docker.
- A Docker Compose file specifies dependencies: DeepStream container, ASP.NET Core container, SQL Server database.
- **Same container runs identically** whether input is Blender video or real IP cameras.
- This ensures that software validated in simulation requires zero code changes for physical deployment—only camera feed source changes.

Simulation-to-Reality Gap:
The primary risk is domain shift: models trained on synthetic Blender data may perform poorly on real RGB camera footage (different texture, lighting, camera lens aberrations). We mitigate this via:

- **Domain Adaptation:** Collecting 10–20% real-world frames to fine-tune (transfer learning).
- **Synthetic Diversity:** Our Blender renderer randomizes lighting, product layout, and the textures extensively.
- **Confidence Thresholding:** In production, we raised the confidence threshold to 0.85 instead of 0.5,which means we only accept only high-confidence detections reducing false positives.

4.2 Economic Constraints

Challenge: Traditional autonomous stores need a lot of money to set up (hardware + real-world data collection).

Our Solution: Digital Twin Strategy

- **Cost Avoidance:** Avoided the cost of collecting data manually, which could have meant hiring people to acquire you enough. We also saved money by not employing other sensors like weight sensors which also needs maintenance.
- **Our Budget:** ~$0 (Blender is free, simulation runs on university lab hardware).
- **Trade-off:** Validation is limited to simulation; real-world robustness is assumed but not tested.

4.3 Sustainability & Environmental Responsibility

Paperless Operations: All receipts are digital, eliminating paper waste.

Energy Efficiency:

- Motion-triggered inference: AI models only process frames when motion is detected in predefined zones, reducing GPU computing.
- Down-side: Yes, the traditional shopping system is better for carbon footprint.

4.4 Ethical Standards And Privacy-by-Design

Privacy Challenge: Video tracking systems make people worry about how long data is kept and how facial recognition can be misused.

How We Do It:

- No facial recognition: The Re-ID module only uses color and texture features of clothing, not face embeddings.
- Only in RAM: Feature vectors and tracking data are only stored in RAM while the session is active.
- When a customer leaves, all tracking data is erased from memory. The only thing that's saved is a events of picking/returning and the invoice.
- Our design follows the rules of GDPR Article 5 (1)(e) (storage limitation), which say that you should store only needed data.
- At the entrance of the store, we will put a sign that says 'this shop uses Computer Vision for the checkout service, no raw videos are saved' which makes customers feel relieved.

4.5 Data Security And API Integrity

Authentication: JWT (JSON Web Token) with HS256 signing.

- Tokens include claims: `{user_id, issue_time, expiration, scopes}`.
- Tokens expire after 8 hours; refresh tokens (stored securely in app local storage) allow renewal.
- Before processing any request, the backend checks the token's signature .

API Security:

- All endpoints need HTTPS (TLS 1.2+).
- Rate limiting: Each user can only make 100 requests/minute (prevents brute-force attacks).
- Input validation: All user inputs (search queries, cart actions) are sanitized and parameterized to stop SQL injection.
- CORS: Backend restricts cross-origin requests to known app domains.

Payment Security:

- The system utilizes a closed-loop **Digital Wallet** architecture.
- ACID-compliant database transactions make sure that transactions are complete.
- **No External Risks:** Since no credit card processing occurs within the prototype, PCI-DSS compliance scope is minimized, and no sensitive financial instrument data is stored.

---

5. Deliverables

5.1 Simulation Environment (Blender Digital Twin)

- 3D Model: A full store space (2×3 meters) with detailed aisle, shelf, and product geometry.
- An automation program that creates training frames with COCO JSON formatted bounding boxes that are automatically labeled.
- Scenario Test Suite: Scripts that run automatically, run Scenarios A–C (occlusion, indecision, crowding, lighting variation), and make performance metrics.
- Documentation: Technical details about the camera's extrinsics, a list of product SKUs, and the parameters for randomization.

5.2 Computer Vision & Tracking Module

- **NVIDIA DeepStream Pipeline Configuration:** We configure the MV3DT Pipeline by using YAML files that specify model paths, camera inputs and so on.
- **Object detection model:** YOLO26 or RF-DETR (will be decided in GP2 after testing both) fine-tuned on synthetic + 5% real-world frames, achieving >90% mAP50 on held-out test set.

- **Re-ID Network:** We can use pretrained models like OSNet That will be work within Deepstream pipeline.
- **Spatial Logic Module:** Custom plugin computing hand-ROI, item interaction events, and homography transformations.
- **Event Stream:** The output will be in the form of JSON event (Customer_ID, Action, Item_ID, Timestamp) sent to message queue (MQTT Broker for example).

5.3 Backend API & Database

- **ASP.NET Core Web API:** RESTful endpoints that handles all use cases such as authentication, products, cart and checkout.
    - Endpoints: POST /auth/register, POST /auth/login, GET /products/search, POST /qr/generate, GET /cart/current, POST /cart/update, POST /checkout/finalize.
    - The response format is JSON with standard error codes (400, 401, 404, 500).
    - Latency SLA: <200ms for 95th percentile requests.
- **SQL Server Database:** Normalized schema (3NF) with 19 primary tables.
    - Indexes on frequently queried columns (UserID, ProductID, SessionID) for optimal query performance.
- **Real-Time Hub:** Integrated ASP.NET Core SignalR Hub**.**

5.4 Mobile Application (Flutter)

- **Functional Features:**
    - User authentication (login, register, password reset).
    - Dynamic QR code display with 30-second refresh.
    - Real-time cart visualization (live item updates).
    - Product search with in-store.
    - Transaction history with digital invoices (PDF rendering).
    - Settings: Theme toggle, language selection (English/Arabic)and setting a budget limit.
- **Platform Support:** iOS (14.0+) and Android (API 25+).

6. Team Roles & Responsibilities

Mohannad Rababah – Computer Vision & Simulation

- Architecture design.
- NVIDIA DeepStream implementation and configuration.
- Custom spatial logic module (hand-ROI computation, homography transformation).
- Model fine-tuning (YOLO26 or RF-DETR).
- Building the digital twin using blender.
- Synthetic data generation.
- Testing accuracy on simulation and real-world.

Laith Allabadi – Hardware Setup & Frontend UI/UX

Primary Responsibilities:

- UI/UX Design.
- Edge-cloud connection.
- Hardware Handling..

Ahmad Edais – Backend & API Integration

Primary Responsibilities:

- ASP.NET Core Web API design and implementation.
- SQL Server database schema design and optimization.
- ASP.NET Core SignalR Hub updating the cart in real time.
- JWT logic for authentication and authorization.
- Adding DeepStream event stream using MQTT Background Service with managing the state of the cart.

Malak Ajlouni – Mobile Application & User Experience

Primary Responsibilities:

Flutter app architecture and UI/UX design with laith that contains:

1. User authentication screens (login, register).
2. QR code display and entry flow.
3. Real-time cart visualization.
4. Product search and detail pages.
5. Transaction history and digital invoice display.
6. Theme and language settings implementation.
7. Digital Wallet interface (balance display and top-up simulation).

7. Project Schedule

Time Frame	Task To Be Completed	Status
September-November	- Identifying system requirements, scope, and implementation platform.	Done
October-November	- Training on the implementation environment.	Done
November-December	- Development and deployment of a simple pilot program	Done
December	- Database designing, drawing the use-case diagram	Done
January	- Report preparation and presentation.	In Progress
February	- Build the simulation environment, generate synthetic data, train an item-detection model on the synthesized data. - build the UI, backend and database.	Planning
March-April	- configuring MV3DT. - test the overall working flow for the project and Check if there're any bugs.	Planning
April-May	- Dissemination of research results.	Planning
May	- Final report and presentation.	Planning

8. Bibliography

[1] Baymard Institute, "46 Cart Abandonment Rate Statistics," *Baymard E-commerce Research*, 2024. [Online]. Available: https://baymard.com/lists/cart-abandonment-rate. [Accessed: Jan 24, 2026].

[2] CropInk, "Cart Abandonment Statistics [2025 Update]," Jan. 21, 2026. [Online]. Available: https://cropink.com/cart-abandonment-statistics. [Accessed: Jan 24, 2026]

[3] Mordor Intelligence, "Smart Retail Market Size, Growth Trends & Research 2030," 2025. [Online]. Available: https://www.mordorintelligence.com/industry-reports/smart-retail-market. [Accessed: Jan 24, 2026].

[4] J.J.L. del Olmo et al., "Computer Vision on the Edge: A Scalable Auto-ID Solution for Retail Automation," Procedia Comput. Sci., vol. 235, pp. 1234–1245, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S187705092502232X. [Accessed: Jan 24, 2026].

[5] T. Sim, S. Baker, and M. Bsat, "The CMU Pose, Illumination, and Expression (PIE) Database," IEEE Trans. Pattern Anal. Mach. Intell., vol. 25, no. 12, pp. 1615–1618, Dec. 2003. [Online]. Available: https://ieeexplore.ieee.org/document/1238571. [Accessed: Jan 24, 2026].

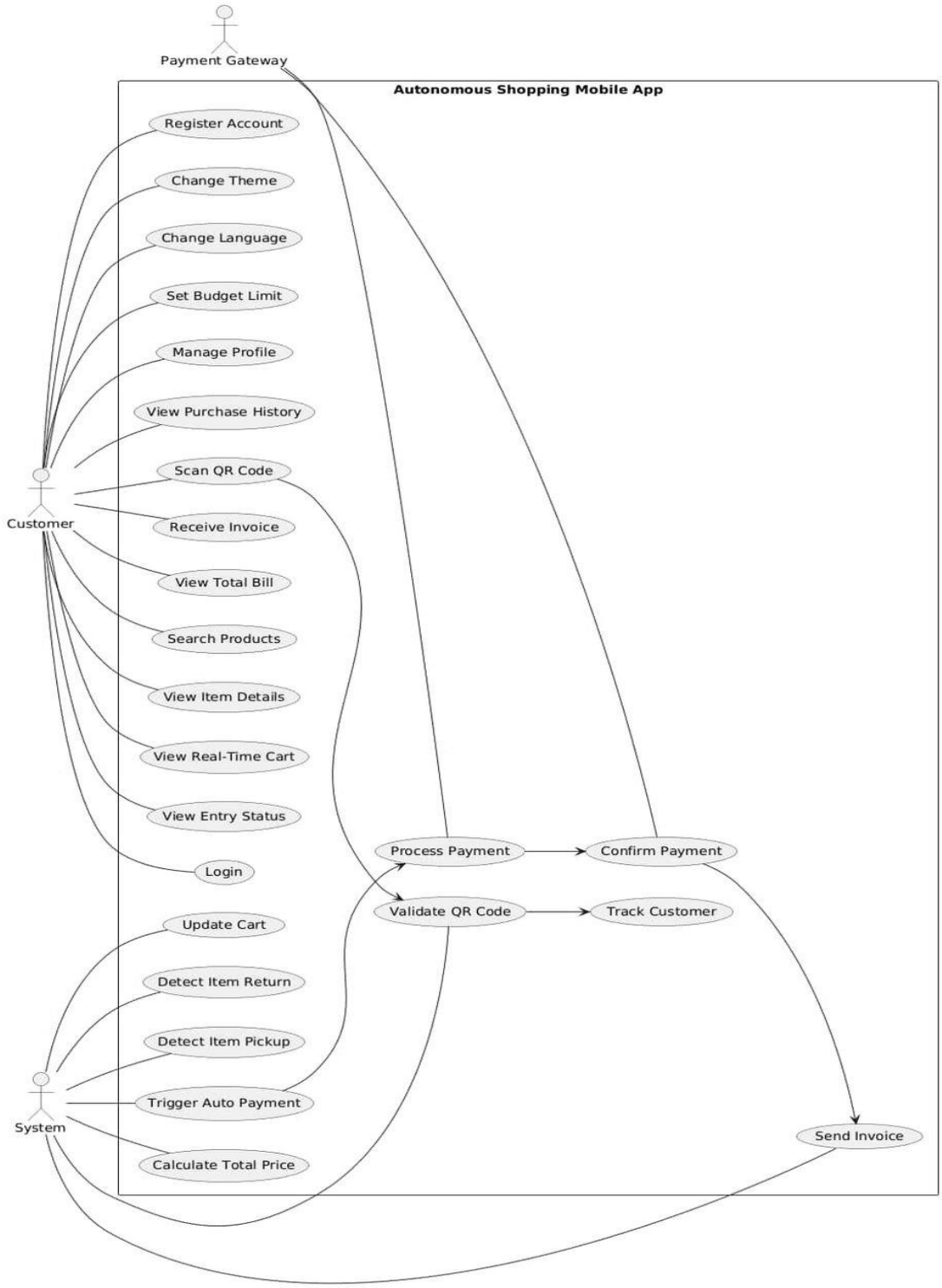
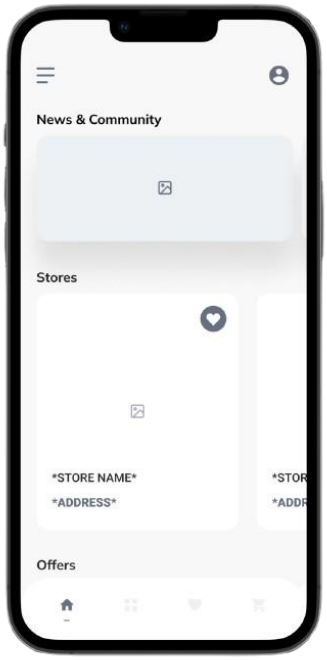
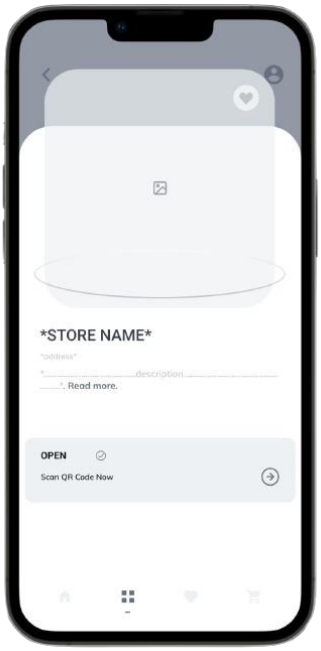
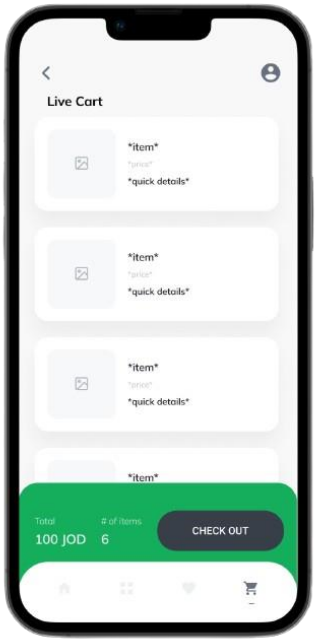
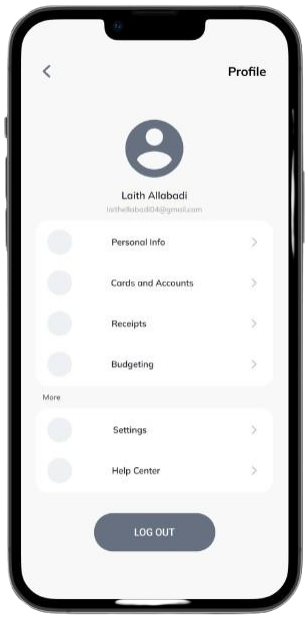
[7] NVIDIA, "DeepStream SDK Development Guide," 2024. [Online]. Available: https://developer.nvidia.com/deepstream-sdk.

[8] Blender Foundation, "Blender 4.0 Reference Manual," 2024. [Online]. Available: https://docs.blender.org/manual/en/latest/.

[9] Ultralytics, "Ultralytics YOLO26," Ultralytics Docs, Jan. 14, 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo26/. [Accessed: Jan 24, 2026].[attached_file:1]

[10] Roboflow, "RF-DETR Object Detection Model: What is, How to Use," 2025. [Online]. Available: https://roboflow.com/model/rf-detr. [Accessed: Jan 24, 2026].[attached_file:2]

Appendix A: Use Case Diagram



Appendix B: Mobile Application UI Mockups